INFORMATION SECURITY LAB · PROJECT REPORT

WireScope

PCAP analysis and threat detection

A web app that turns a packet capture into an indexed SQLite database and an XML file, runs five threat-detection rules over it, and shows the results on one interactive dashboard

Course	Information Security Lab (IT_3121)
Lab modules	PCAP Analysis (PCAP → SQLite, PCAP → XML) Traffic Analysis with PyShark
Student	Savya Raj 245811454
Programme	B.Tech Information Technology, 3rd year
Institution	Manipal Institute of Technology, Bengaluru
Live app	wirescope.onrender.com
Source code	github.com/savya14/wirescope
Report date	25 October 2026

Every capture analysed in this report is synthetic, generated with Scapy; no real user traffic was captured. All results were measured on 25 October 2026 from the code in the repository at that date, with tshark 4.2.2, PyShark 0.6 and Scapy 2.8.

Contents

1 Summary **2**

2 Objectives and lab requirements **3**
2.1 Goals beyond the brief . . . 3

3 System design **4**
3.1 Architecture . . . 4
3.2 The pipeline . . . 4
3.3 Technology . . . 5

4 PCAP to SQLite and XML **5**
4.1 The packets table . . . 5
4.2 The XML export . . . 5
4.3 Downloads . . . 6

5 Traffic analysis with PyShark **7**
5.1 How the parser works . . . 7
5.2 The Scapy fallback . . . 7
5.3 Comparing the two engines . . . 7

6 Threat detection **8**
6.1 The five rules . . . 8
6.2 Example: the port-scan rule . . . 8
6.3 Severity and evidence . . . 8

7 The dashboard **10**
7.1 Layout . . . 10
7.2 Cross-filtering . . . 12
7.3 REST API . . . 13

8 Testing and results **13**
8.1 Test captures . . . 13
8.2 Method . . . 14
8.3 Results . . . 14
8.4 Other checks . . . 15
8.5 A check on ordinary traffic . . . 15

9 Deployment and security **16**
9.1 Build and hosting . . . 16
9.2 Security of the app itself . . . 16

10 Limitations and future work **17**
10.1 Weaknesses found in testing . . . 17
10.2 Other limitations . . . 17
10.3 Future work . . . 17

11 Conclusion **18**

A Repository map **19**

B Running it locally **19**

References **20**

1 Summary

Wireshark shows a capture one packet at a time. That is the right view for reading a single frame, but a poor one for spotting a port scan spread over forty packets or a host quietly sending most of the traffic. **WireScope** is a web app built for the Information Security Lab [1] that takes a `.pcap` or `.pcapng` file and answers those capture-wide questions.

It parses every packet with **PyShark** (Wireshark's `tshark` engine), with **Scapy** as an automatic fallback, and normalises each one into ten fields. The packets go into an indexed **SQLite** table and an **XML** file. Five SQL rules then look for port scans, plaintext protocols, backdoor ports, heavy talkers and ICMP floods. A React dashboard shows the findings, eight charts and a searchable packet table.

- **Both lab modules are covered.** PCAP → SQLite and PCAP → XML (Section 4), and traffic analysis with PyShark (Section 5).
- **The detections are correct on all four test captures.** Each preset capture raised exactly the alerts it was built to raise: 5, 1, 2 and 0. The benign capture raised none (Section 8).
- **The two parsers agree.** On 2,737 packets across five captures, PyShark and Scapy produced identical values in all ten fields. Scapy was about 15 to 25 times faster (Section 5.3).
- **It is deployed.** One Docker container on Render serves both the API and the dashboard. Each visitor's data is kept in separate files that are deleted after an hour (Section 9).
- **Testing also found four weaknesses in the rules.** The serious one: a client making 20 ordinary HTTPS connections to one server gets the server flagged as a HIGH-severity port scanner. The others inflate alerts that should fire anyway. Section 10 lists all four with fixes.

2 Objectives and lab requirements

The lab asked for two modules. WireScope implements both and adds automated detection and a dashboard on top.

Table 1. Lab requirements and where WireScope meets them

Requirement	How WireScope meets it	Code
PCAP → SQLite	Every frame becomes one row of ten fields in a `packets` table, bulk-inserted with five indexes	`parser.py`: `load_to_sqlite`
PCAP → XML	The same rows are written as a `<capture>` tree with one `<packet>` element per frame	`parser.py`: `export_to_xml`
Traffic analysis with PyShark	Captures are read through `pyshark.FileCapture`, which drives `tshark`; Scapy takes over if `tshark` is missing or fails	`parser.py`: `parse_pcap`
Analysis of the traffic	Five SQL detection rules, eight charts and a filterable packet explorer	`detection.py`, `backend/main.py`, `frontend/`

2.1 Goals beyond the brief

- **Explain every alert.** Each alert carries the packets or hosts that triggered it, and one click filters the packet table to exactly that evidence.
- **Work without Wireshark installed.** If `tshark` is absent, the Scapy parser produces the same ten fields, so the rest of the pipeline is unchanged.
- **Be testable.** Four synthetic captures with known contents let anyone check the detections against an expected answer.
- **Be usable by several people at once.** The public deployment keeps each browser's capture separate.

3 System design

WireScope is one Python service and one single-page web app. The service parses the capture, stores it, runs the rules and computes the chart data; the browser only displays what it receives.

3.1 Architecture

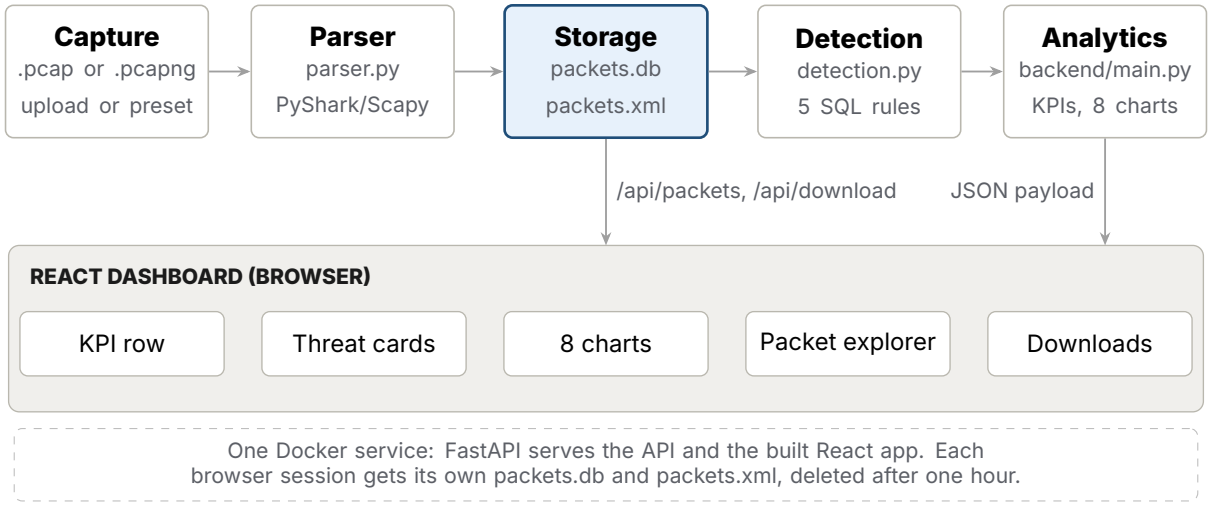


Figure 1. WireScope's architecture. The capture is parsed once; the database is then the single source for the alerts, the charts, the packet table and the downloads.

3.2 The pipeline

Loading a capture, by upload or by picking a preset, runs the same six steps on the server:

1. **Receive.** The file is checked for type (`.pcap`, `.pcapng`, `.cap`) and size (25 MB by default), and saved to a temporary file.
2. **Parse.** `parse_pcap` reads up to 20,000 packets with PyShark, or with Scapy if `tshark` is unavailable or raises an error.
3. **Store.** The rows are bulk-inserted into SQLite and written to XML, under the browser's session id.
4. **Detect.** The five rules run as SQL queries against that database.
5. **Aggregate.** Ten more SQL queries compute the headline numbers and the data for the eight charts.
6. **Respond.** One JSON payload goes back to the browser. The packet explorer and the downloads query the same database afterwards.

3.3 Technology

Table 2. Technology used

Layer	Choice
Packet dissection	PyShark 0.6 over `tshark` (Wireshark 4.2.2 in testing); Scapy 2.8 as fallback
Storage	SQLite 3 [5], one database and one XML file per session
Detection	SQL queries read into pandas data frames
API	FastAPI [6] with Pydantic response models, served by Uvicorn
Dashboard	React 19, TypeScript 6, Vite 8, Tailwind CSS 4, Recharts 3, IBM Plex fonts
Test data	Scapy scripts that write four reproducible scenario captures
Deployment	Two-stage Docker build on Render's free tier

4 PCAP to SQLite and XML

Every packet is reduced to the same ten fields, whichever parser read it. That fixed shape is what lets one SQL schema, one XML format and one set of rules serve any capture.

4.1 The packets table

Listing 1. The table created by `load_to_sqlite`

```
CREATE TABLE packets (
    id           INTEGER PRIMARY KEY AUTOINCREMENT,
    packet_no    INTEGER,   -- frame number in the capture
    timestamp    REAL,      -- capture time, Unix seconds
    src_ip       TEXT,      -- IPv4 / IPv6 source, or ARP sender
    dst_ip       TEXT,
    src_port     INTEGER,   -- TCP and UDP only
    dst_port     INTEGER,
    protocol     TEXT,      -- TCP, UDP, ICMP, ARP or OTHER
    app_protocol TEXT,      -- e.g. TLS, DNS, FTP, METASPLOIT
    tcp_flags    TEXT,      -- decoded, e.g. SYN, ACK-RST
    length       INTEGER    -- frame length in bytes
);
```

The rows are inserted in one `executemany` call inside a single transaction, which is far faster than one insert per packet. Five indexes then speed up the queries the rules and the explorer run most: on `src_ip`, `dst_ip`, `protocol`, `dst_port` and `timestamp`.

Fields a packet does not have are stored as `NULL` rather than guessed. An ARP frame has no ports, an ICMP packet no TCP flags. The rules filter on `IS NOT NULL` where it matters, so a missing field never counts as a match.

4.2 The XML export

The same rows are written as an indented XML tree. Empty fields stay as empty elements, so every `<packet>` has the same ten children and the file is easy to validate or read back.

Listing 2. The first packet of `full_attack.pcap` in `packets.xml`

```
<?xml version='1.0' encoding='utf-8'?>
<capture total_packets="602">
  <packet id="1">
    <packet_no>1</packet_no>
    <timestamp>1714000000.0</timestamp>
    <src_ip>192.168.1.10</src_ip>
    <dst_ip>192.168.1.1</dst_ip>
    <src_port />
    <dst_port />
    <protocol>ARP</protocol>
    <app_protocol>ARP</app_protocol>
    <tcp_flags />
    <length>42</length>
  </packet>
```

4.3 Downloads

The dashboard footer offers three files for offline work. For the 602-packet full attack capture they measured:

Table 3. Downloadable artifacts for `full_attack.pcap` (602 packets)

File	Size	Use
`packets.db`	124.0 KB	Open in any SQLite client and run further queries
`packets.xml`	205.2 KB	The PCAP → XML deliverable
`packets.csv`	45.7 KB	Generated on request from the database, for spreadsheets

5 Traffic analysis with PyShark

PyShark [2] is a Python wrapper around `tshark` [3], the command-line form of Wireshark. It gives WireScope Wireshark's full set of protocol dissectors without reimplementing any of them.

5.1 How the parser works

1. **Find tshark.** `find_tshark` checks the `PATH` and the usual install folders on macOS and Linux, and reads the version for the dashboard badge.
2. **Stream the capture.** `pyshark.FileCapture` runs with `keep_packets=False`, so packets are read one at a time and not held in memory. A fresh event loop is created first, because PyShark is asynchronous and the web server calls it from a worker thread.
3. **Read each layer.** Addresses come from the IP, IPv6 or ARP layer; ports and flags from TCP or UDP. A missing or malformed field becomes `None` instead of stopping the parse.
4. **Name the application protocol.** The highest layer tshark dissected is used when it is meaningful (for example `DNS` or `TLS`). Otherwise the parser looks for HTTP, FTP, Telnet, SSH, DNS or TLS among the layers, and finally maps the port through a table of 30 well-known ports, including the backdoor ports 4444, 31337, 6667 and 1337.
5. **Decode TCP flags.** Hex values, integers and Scapy's letter codes all become the same readable form, such as `SYN`, `SYN-ACK` or `ACK-RST`.
6. **Stop at 20,000 packets.** Larger captures are cut there, and the dashboard says so (*Capped at 20,000*).

5.2 The Scapy fallback

If `tshark` is not installed, or PyShark raises an error, `parse_pcap` logs the error and reads the file again with Scapy's [4] `PcapReader`. Scapy is pure Python, so it needs nothing outside `pip`. It names application protocols from its own DNS layer and the same port table. The header badge always shows which engine produced the current data, for example *PyShark* (*tshark 4.2.2*) or *Scapy fallback*.

5.3 Comparing the two engines

Each bundled capture was parsed with both engines and the ten fields compared packet by packet.

Table 4. PyShark against Scapy on every bundled capture (25 October 2026)

Capture	Size	Packets	PyShark	Scapy	Identical rows
`full_attack.pcap`	272.4 KB	602	3.05 s	1.32 s*	602 / 602
`port_scan.pcap`	85.7 KB	557	2.71 s	0.14 s	557 / 557
`malware_c2.pcap`	82.4 KB	503	2.38 s	0.13 s	503 / 503
`normal.pcap`	81.3 KB	473	2.96 s	0.21 s	473 / 473
`sample.pcap`	272.2 KB	602	3.97 s	0.15 s	602 / 602

*The first Scapy run includes loading Scapy itself, about 1.2 s.

All 2,737 packets matched in every field, so the fallback changes nothing downstream. PyShark was 15 to 25 times slower, because it starts `tshark` as a separate process and parses its output. The trade is still worth making: on real traffic, tshark's dissectors can recognise a protocol on a non-standard port, which the port table cannot. These synthetic captures use standard ports, so they do not show that difference.

6 Threat detection

Each rule is one SQL query, plus a little pandas for thresholds and wording. A rule that fires returns a title, a severity, a one-line summary and its evidence: the packets or hosts behind it.

6.1 The five rules

Table 5. Detection rules. The ATT&CK column is the closest MITRE technique [7].

Rule	Severity	Fires when	ATT&CK
Port scan	HIGH	One source sends SYN-only packets to at least 15 distinct ports on one destination. If no pair qualifies, the query is repeated without the flag condition.	T1046 Network Service Discovery
Plaintext protocols	MEDIUM	Any packet uses port 21, 23, 25, 80, 110 or 143 in either direction, or is identified as FTP, HTTP, Telnet, POP3, IMAP or SMTP.	T1040 Network Sniffing
Backdoor / C2 ports	MEDIUM	Any packet uses port 4444 (Metasploit), 31337 (Back Orifice), 6667 (IRC botnets) or 1337.	T1571 Non-Standard Port
Heavy talker	LOW	A source is at or above the 95th percentile of bytes sent, and the top source sends at least 40% of all bytes.	T1048 Exfiltration Over Alternative Protocol (possible)
ICMP volume	LOW	A source sends at least 15 ICMP packets. If none does, any source at or above max(5, 2.5 × the average) is flagged.	T1018 Remote System Discovery; T1498 Network Denial of Service

6.2 Example: the port-scan rule

A SYN scan sends a connection request to many ports and never completes the handshake. The query counts distinct destination ports per source–destination pair, using only packets with SYN set and ACK clear:

Listing 3. The port-scan query in `detection.py`

```
SELECT src_ip, dst_ip,
       COUNT(DISTINCT dst_port) AS distinct_ports,
       COUNT(*)                 AS total_probes,
       MIN(timestamp) AS scan_start, MAX(timestamp) AS scan_end
FROM packets
WHERE dst_port IS NOT NULL AND src_ip IS NOT NULL AND dst_ip IS NOT NULL
  AND tcp_flags LIKE '%SYN%' AND tcp_flags NOT LIKE '%ACK%'
GROUP BY src_ip, dst_ip
HAVING distinct_ports >= 15
ORDER BY distinct_ports DESC;
```

Requiring SYN without ACK means a busy server answering many clients is not mistaken for a scanner, since its replies carry SYN-ACK. If no pair qualifies, the rule repeats the query without the flag condition, meant for UDP scans. Section 10 shows that this fallback misfires on ordinary traffic.

6.3 Severity and evidence

- **Severity** is fixed per rule: HIGH for active reconnaissance, MEDIUM for exposure or backdoor activity, LOW for volume anomalies that need a human to judge. The dashboard sorts alerts by severity and colours each one.

- **Evidence** is a table attached to each alert: up to 100 packets for packet-level rules, or the flagged hosts with their byte counts and traffic share for the heavy-talker rule.
- **Investigate** filters the packet explorer to that evidence. The server re-runs the rule and matches its packet numbers, or its hosts for host-level alerts.

7 The dashboard

The dashboard is built so that an analyst can go from the headline numbers to the packets behind an alert in two clicks.

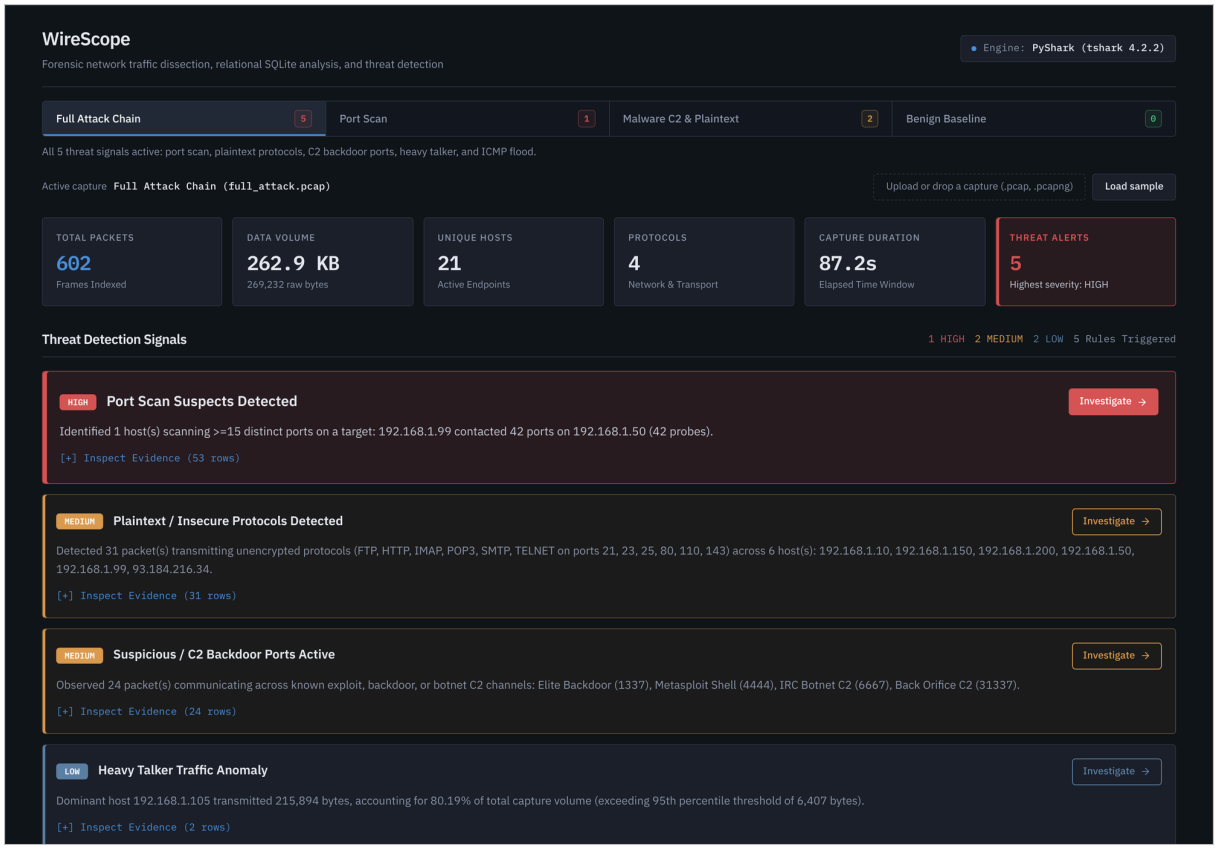


Figure 2. The top of the dashboard with the Full Attack Chain loaded: scenario tabs, the engine badge, six headline numbers and the alerts sorted by severity.

7.1 Layout

- **Header and scenarios.** The engine badge, four preset scenario tabs showing their expected alert counts, an upload box that accepts drag and drop, and the name of the active capture.
- **Six headline numbers.** Total packets, data volume, unique hosts, protocols, capture duration and threat alerts. The alert card takes the colour of the highest severity, or turns green when the capture is clean.
- **Threat cards.** One per alert, each with an *Investigate* button and an expandable evidence table (Figure 3).
- **Eight charts.** Protocol mix, top talkers, top destination ports, a port-scan scatter, a traffic timeline, packet sizes, a host-to-host flow diagram and a conversation matrix (Figures 4 and 5).
- **Packet explorer.** A paginated table, 50 rows a page, with protocol, host and port filters, served by SQL queries on the server (Figure 6).
- **Downloads.** `packets.db`, `packets.xml` and `packets.csv`.

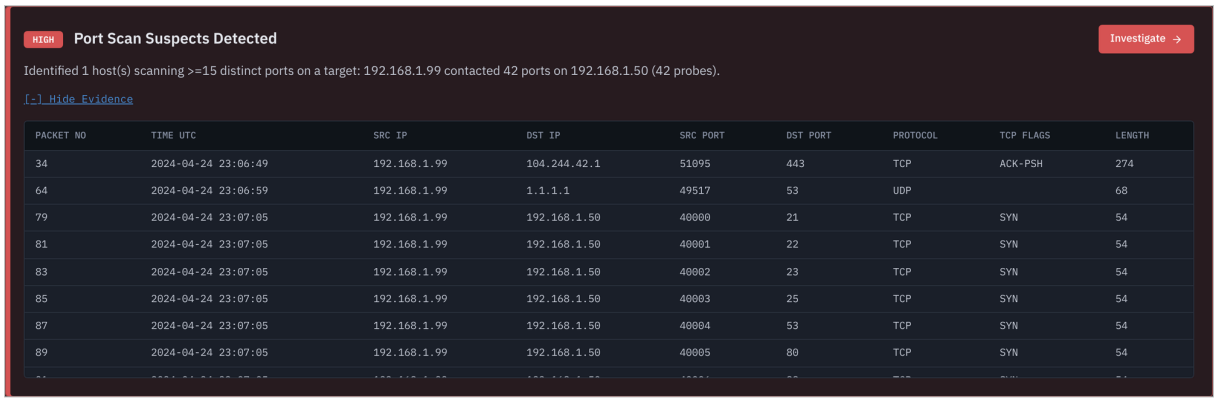


Figure 3. The port-scan alert with its evidence opened: 192.168.1.99 sending SYN probes to one port after another on 192.168.1.50.

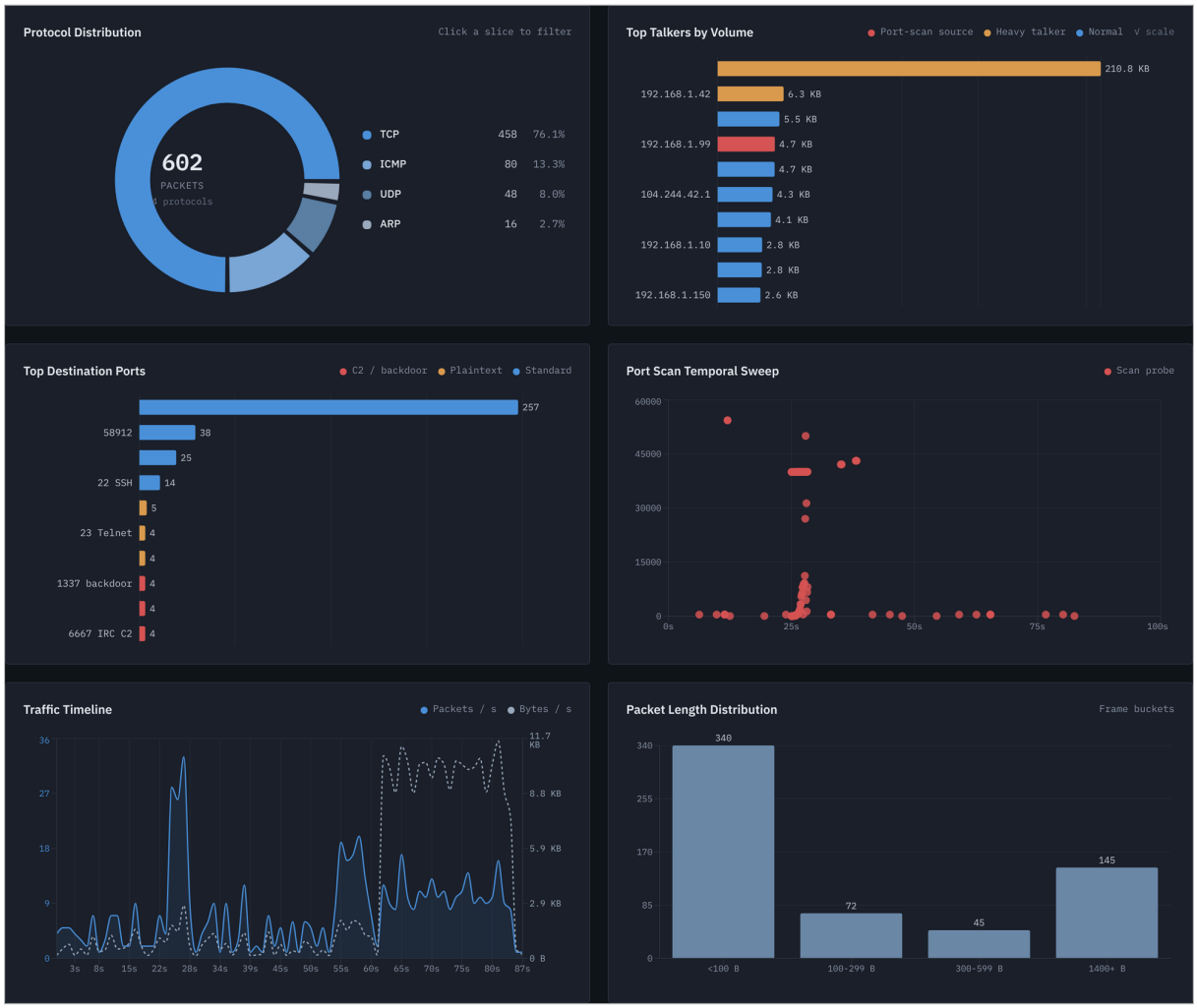


Figure 4. Six of the eight charts. Threat hosts and ports keep their alert colour: red for the port-scan source and backdoor ports, amber for the heavy talker and plaintext ports.

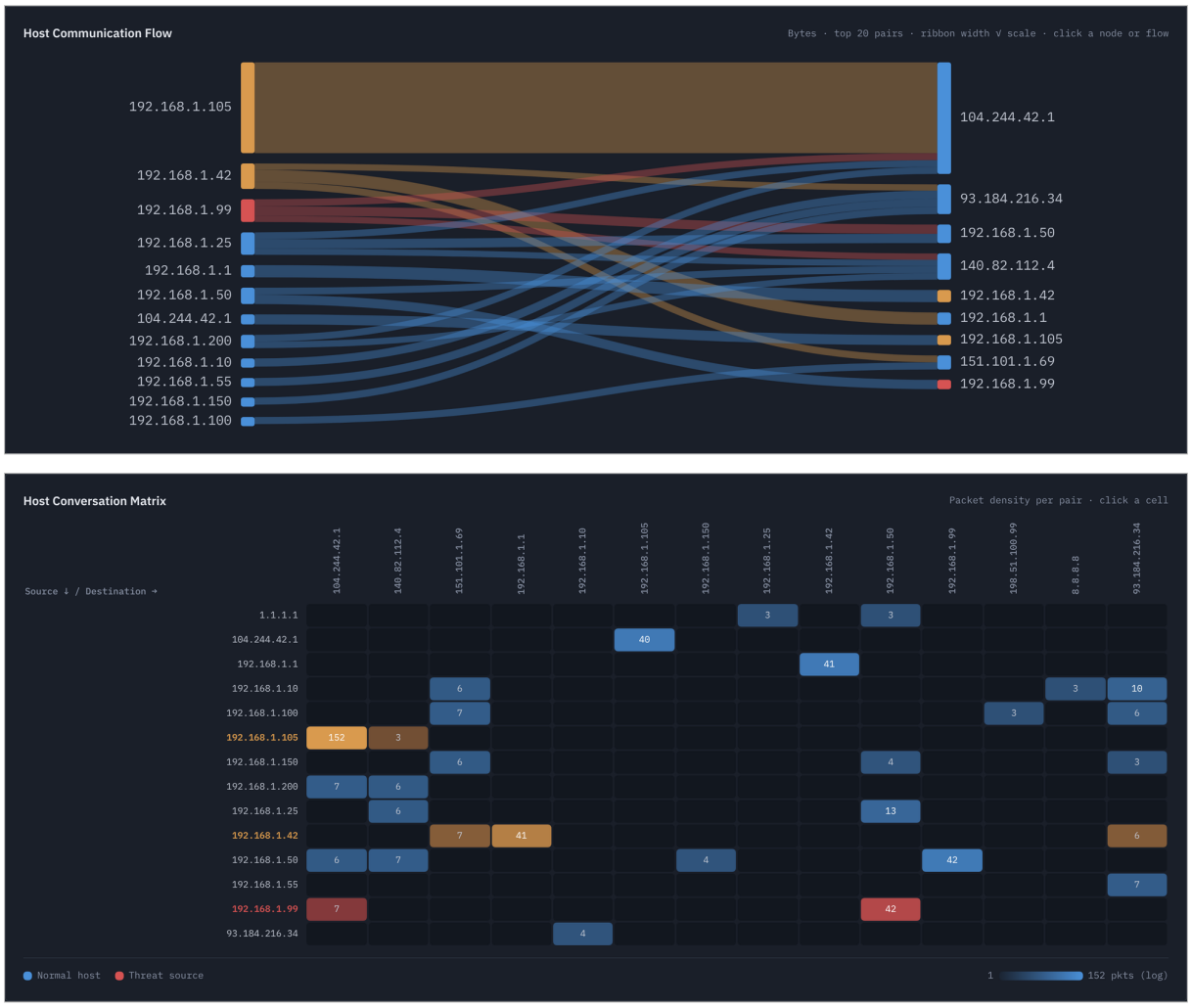


Figure 5. Who talks to whom. Top: bytes between host pairs, where the heavy talker 192.168.1.105 dominates. Bottom: packets per pair, with threat sources named in their alert colour.

7.2 Cross-filtering

The charts, the alerts and the packet table share one filter. Clicking a protocol slice, a host bar, a port, a flow ribbon or a matrix cell filters the packet explorer and scrolls to it. Clicking the same element again clears that filter. Active filters appear as chips that can be removed one at a time.

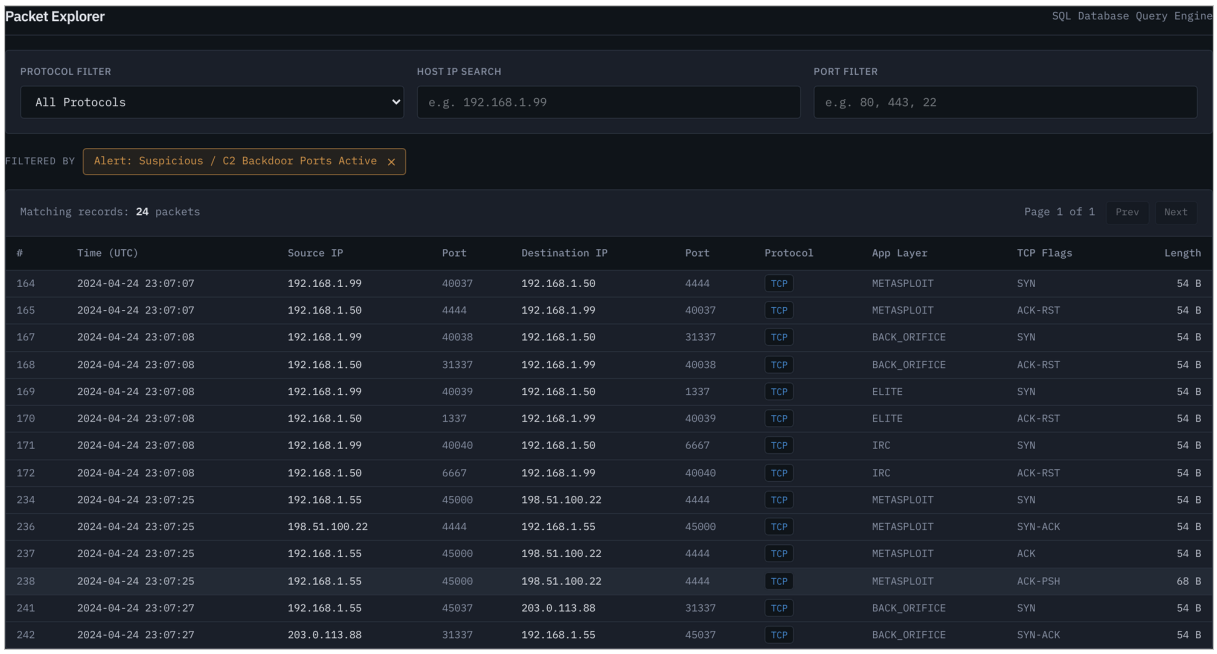


Figure 6. *Investigate* on the backdoor-port alert: the explorer shows only its 24 evidence packets.

7.3 REST API

Table 6. API endpoints. All except `/api/scenarios` need an `X-Session-Id` header.

Method	Endpoint	Returns
GET	`/api/health`	Parser engine, packet count, whether the session database is ready
GET	`/api/scenarios`	The four preset scenarios and their expected alert counts
GET	`/api/scenario/{id}`	Runs the pipeline on a preset capture; full dashboard payload
GET	`/api/sample`	The same for the bundled `sample.pcap`
POST	`/api/analyze`	The same for an uploaded capture
GET	`/api/packets`	One page of packets; filters: protocol, search, port, host, src, dst, alert
GET	`/api/download/{file}`	`packets.db`, `packets.xml` or `packets.csv`

8 Testing and results

Every preset capture raised exactly the alerts it was designed to raise, and the benign capture raised none.

8.1 Test captures

Real captures cannot be shared and have no answer key, so the project generates its own with Scapy. Each one mixes the same benign background traffic with a known set of attacks.

Table 7. The four preset captures

Scenario	Packets	Tag	What it contains
Benign Baseline	473	CLEAR	ARP, DNS, HTTPS, one SSH session and five ping pairs: the background traffic used in every scenario
Port Scan	557	HIGH	Background plus a SYN scan of 42 ports on 192.168.1.50 from 192.168.1.99
Malware C2 & Plaintext	503	MEDIUM	Background plus HTTP, FTP and Telnet sessions and four backdoor channels on 4444, 31337, 6667 and 1337
Full Attack Chain	602	CRITICAL	All of the above, a bulk stream of large packets from 192.168.1.105, and 40 pings from 192.168.1.42 to the gateway

8.2 Method

Each scenario was loaded through the real API endpoint, the same call the dashboard makes, with `tshark` 4.2.2 installed. The alerts returned were compared with the expected count, and the packet explorer, downloads and error handling were checked on the full attack capture.

8.3 Results

Table 8. Alerts raised per scenario (PyShark engine)

Scenario	Expected	Raised	Alerts	Hosts	Time
Benign Baseline	0	0	None	15	2.5 s
Port Scan	1	1	Port scan (HIGH)	16	2.2 s
Malware C2 & Plaintext	2	2	Plaintext, backdoor ports (both MEDIUM)	19	1.9 s
Full Attack Chain	5	5	1 HIGH, 2 MEDIUM, 2 LOW	21	2.7 s

Time is the full request: parse, store, detect and aggregate.

Table 9. What each alert reported on the Full Attack Chain

Alert	Evidence	Finding
Port scan	53 rows	192.168.1.99 sent SYN probes to 42 distinct ports on 192.168.1.50
Plaintext	31 rows	FTP, HTTP, IMAP, POP3, SMTP and Telnet traffic across six hosts
Backdoor ports	24 rows	Traffic on 1337, 4444, 6667 and 31337
Heavy talker	2 hosts	192.168.1.105 sent 215,894 bytes, 80.19% of the capture; the 95th percentile was 6,407 bytes
ICMP volume	80 rows	192.168.1.42 and 192.168.1.1 each sent 40 ICMP packets to one host

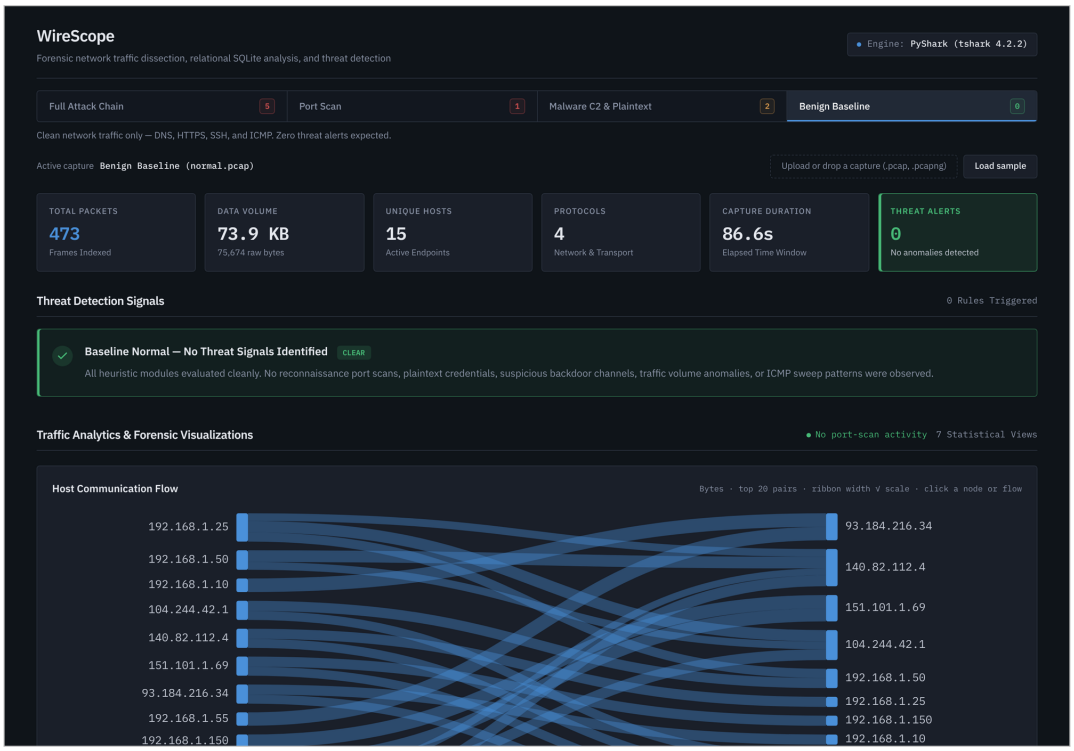


Figure 7. The Benign Baseline: zero alerts, a green status card, and the port-scan chart reporting no scan activity.

8.4 Other checks

- **Packet explorer.** On the full attack capture, the port-scan alert filter returned 53 packets, the ICMP protocol filter 80, and the port filter 4444 returned 6.
- **Downloads.** All three files downloaded with the correct content types.
- **Bad input.** A request without a session id was refused with HTTP 400. A `.txt` upload was refused with *Unsupported file format '.txt'*.
- **Build.** The TypeScript type check and the production build both passed.

8.5 A check on ordinary traffic

The preset captures contain nothing a rule could mistake for an attack, so one more capture was built: a single client opening 20 normal HTTPS connections to one web server, 120 packets in all. Nothing in it is malicious.

Table 10. Alerts raised on 20 ordinary HTTPS connections from one client to one server

Alert	Severity	What it reported
Port scan	HIGH	"151.101.1.69 contacted 20 ports on 10.0.0.5 (40 probes)": the web server, flagged for replying to the client's 20 source ports
Heavy talker	LOW	The web server sent 70.79% of the bytes

Both are false alarms. The SYN-only query correctly found no scan, so the rule fell back to counting every packet, and the server's replies to 20 different client ports looked like probes. Section 10 explains the fix.

9 Deployment and security

WireScope runs as one Docker container, so the public site, the API and the parser share one address and need no cross-origin setup.

9.1 Build and hosting

- **Stage 1** uses Node 22 to build the React app into static files.
- **Stage 2** starts from Python 3.12, installs `tshark` without packet capture rights (it only reads files), installs the Python packages, copies the code, captures and built app, and runs Uvicorn as an unprivileged user.
- **Render** builds the image from `render.yaml` on the free plan, with `/api/scenarios` as the health check. Free services sleep when idle, so the first request after a quiet period takes a few seconds.

Table 11. Settings

Variable	Default	Purpose
`MAX_UPLOAD_MB`	25	Largest accepted upload
`SESSION_TTL_SECONDS`	3600	How long a session's files are kept
`SESSIONS_DIR`	`data/sessions`	Where session files are written
`ALLOWED_ORIGINS`	localhost	Only needed if the dashboard is hosted elsewhere
`PORT`	8000	Set by the host

9.2 Security of the app itself

A tool that parses files from strangers is itself an attack surface. These are the controls in the code:

Table 12. Risks and controls

Risk	Control
One visitor sees another's capture	Each browser has a random session id; files are stored per id and deleted after the TTL
Path traversal through the session id	The id must match `[A-Za-z0-9_-]{8,64}`, so it cannot contain `/` or `..`
SQL injection through filters	Every filter value is passed as a bound parameter, never pasted into the query
Oversized or wrong uploads	At most 25 MB is read; only capture extensions are accepted; a capture that fails to parse returns HTTP 400
A flaw in a Wireshark dissector	tshark runs as an unprivileged user without capture rights, inside the container
Calls from other websites	Cross-origin requests are limited to the configured origins

What is not covered: there is no login and no rate limit, the file type is checked by extension rather than content, and uploads are processed on a shared server. The live site should not be given captures that contain sensitive traffic.

10 Limitations and future work

Testing found four problems. The first raises a HIGH alert on ordinary traffic and should be fixed before the tool is used on real captures. The other three do not change which alerts fire on the test data, but they inflate the evidence and would add noise on real traffic.

10.1 Weaknesses found in testing

- **The port-scan fallback flags busy servers.** When the SYN-only query finds nothing, which is the normal case for clean traffic, the rule counts every packet instead. A server's replies go to each client connection's own source port, so 20 connections look like a 20-port scan by the server (Section 8.5). *Fix:* use the fallback only when the capture has no TCP flags at all, and only for UDP; ignore packets whose source port is a well-known service port.
- **Backdoor and plaintext rules count scan probes.** The rules match any packet on a listed port, even a refused connection. In the full attack, 8 of the 24 backdoor-port packets are the port scan's probes and the target's RST replies. SMTP, POP3 and IMAP appear in the plaintext alert only because the scan touched ports 25, 110 and 143. *Fix:* count only connections that completed a handshake, or exclude hosts already flagged as scanners.
- **The ICMP rule flags the host that replies.** The gateway 192.168.1.1 is flagged alongside the pinging host, because its 40 echo replies are counted too. The alert also calls 40 pings to a single host a sweep, when a sweep spans many hosts. *Fix:* count only echo requests, and label one destination as a flood and many as a sweep.
- **Port-scan evidence is too broad.** The evidence lists every packet from the scanning host, including its normal HTTPS and DNS traffic: 53 rows for 42 probes. *Fix:* restrict the evidence to the flagged source–destination pair.

10.2 Other limitations

- **Synthetic data only.** The rules have been checked against known answers, not measured on real traffic, so there are no precision or recall figures.
- **Fixed thresholds.** 15 ports, 15 ICMP packets and a 40% traffic share are hard-coded. A slow scan, or one spread over several targets, stays under them. With only a few hosts, the top sender is above the 95th percentile almost by definition, so the heavy-talker alert rests on the 40% share alone.
- **Port-based rules.** HTTP on port 8080 is not flagged as plaintext, and an encrypted service on port 4444 would be.
- **Capacity.** Captures are cut at 20,000 packets and 25 MB, and a session's details are kept in memory, so a server restart loses them.
- **ATT&CK labels.** The technique mapping in Table 5 lives in the documentation and is not yet shown on the alert cards.

10.3 Future work

1. Fix the four weaknesses above, starting with the port-scan fallback, and add a test capture for each so they stay fixed.
2. Measure the rules on labelled public captures to get real false-alarm rates.
3. Make thresholds configurable per capture, with sensible defaults.
4. Use more of what tshark dissects: TLS server names, DNS query names and HTTP hosts.
5. Show the ATT&CK technique on each alert card.

11 Conclusion

WireScope meets both lab modules. Captures go into SQLite and XML in a fixed ten-field form, and PyShark does the traffic analysis with Scapy as a fallback that gives identical results. On top of that, five SQL rules, eight charts and a filterable packet table turn a capture into an investigation that starts from the alert and ends at the packets.

The tests show the rules raise exactly the expected alerts on all four scenarios. They also show where the rules are too loose. The port-scan fallback flags a busy web server, scan probes count as backdoor traffic, and replies count as pings. Fixing those, and testing on real captures, is the next step.

A Repository map

Table 13. Source files and their roles (lines of code)

File	Lines	Role
`parser.py`	482	PyShark and Scapy parsers, SQLite load, XML export
`detection.py`	379	The five detection rules
`backend/main.py`	832	FastAPI app: sessions, endpoints, chart queries
`generate_sample.py`	418	Writes `sample.pcap`, the full attack capture
`generate_scenarios.py`	354	Writes the four scenario captures
`record_demo.py`	209	Records the demo video with Playwright
`capture_screenshots.py`	170	Captures README screenshots
`frontend/src/` (18 files)	2,370	React dashboard: header, scenarios, KPIs, alerts, charts, explorer
`Dockerfile`, `render.yaml`	–	Two-stage image and Render blueprint
`scenarios/`, `sample.pcap`	–	Bundled test captures

B Running it locally

Listing 4. Setup and launch

```
git clone https://github.com/savya14/wirescope.git && cd wirescope
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
cd frontend && npm install && cd ..
python generate_scenarios.py      # writes the four test captures
./run_all.sh                      # API on :8000, dashboard on :5173
```

`tshark` is optional: without it, the header badge reads *Scapy fallback* and everything else works the same.

References

[1] S. Raj. *WireScope* source code. `https://github.com/savya14/wirescope`. Live app: `https://wirescope.onrender.com`.

[2] KimiNewt. *PyShark: Python packet parser using Wireshark's dissectors*. `https://github.com/KimiNewt/pyshark`.

[3] Wireshark Foundation. *tshark manual page*. `https://www.wireshark.org/docs/man-pages/tshark.html`.

[4] Scapy project. *Scapy documentation*. `https://scapy.readthedocs.io/`.

[5] SQLite. *SQLite documentation*. `https://www.sqlite.org/docs.html`.

[6] FastAPI. *FastAPI documentation*. `https://fastapi.tiangolo.com/`.

[7] MITRE. *ATT&CK Enterprise techniques* T1046, T1040, T1571, T1048, T1018 and T1498. `https://attack.mitre.org/techniques/enterprise/`.